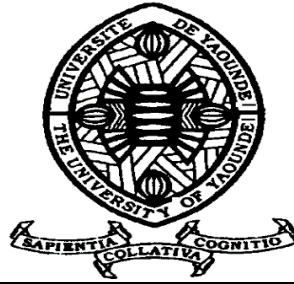


République du CAMEROUN

Paix – Travail – Patrie

Faculté des Sciences

Département d'informatique



Republic of CAMEROUN

Peace – Work – Fatherland

Faculty of Sciences

Computer Science Department

UE : INF 462

**TP 3: API GATEWAY, AUTHENTICATION AND
AUTHORIZATION**

GROUPE 5

MOUSSA ABAKAR ABBAZENE

23V2834

ABDELAZIZ MAHAMAT LOUKY

18T2916

SEMDI HONORE

14L1992

MESSI DZOU SYLVESTRE CEDRIC

23V2853

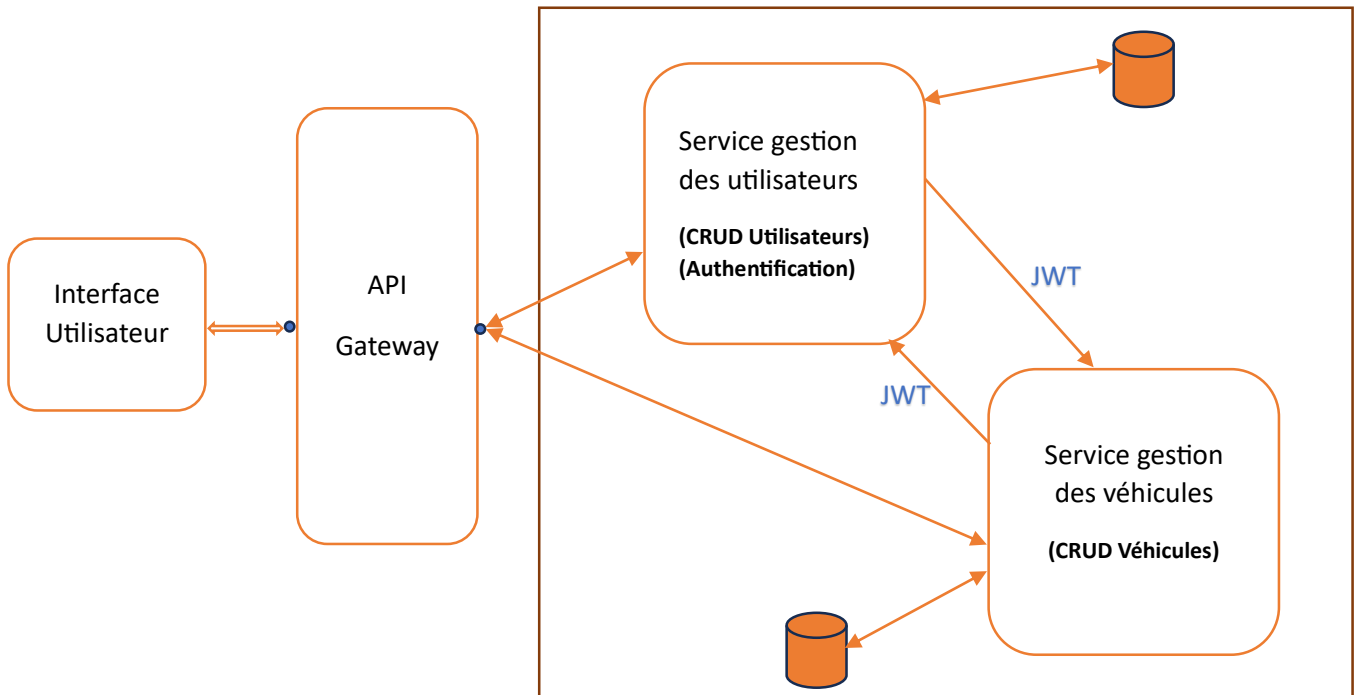
Enseignant : Dr. KIMBI XAVIERA

Sous la supervision de : M. ATEMENGUE REGIS

Année académique : 2023-2024

Taches :

1. Proposer la nouvelle architecture d'API avec les services de la gestion des utilisateurs et L'API Gateway



1. Comment peut-on sécuriser les micro services de cette API

Pour sécuriser les micro services de notre API nous pouvons utiliser comme méthode :

- ✚ Authentification : Utilisation de JSON Web Token (JWT) pour l'authentification des utilisateurs. Un utilisateur doit fournir un token JWT valide pour accéder aux ressources.
- ✚ Autorisation : Basée sur des rôles, où les accès aux différentes parties de l'API sont restreints selon les rôles de l'utilisateur.

2. Liste des End Points de l'API

- ✚ Create User: POST /api/users
- ✚ Update User: PUT /api/users/{id}
- ✚ Login: POST /api/auth/login
- ✚ Refresh Token: POST /api/auth/refresh

3. Concepts d'Authentification et d'Autorisation

Authentification : L'authentification est le processus de vérification de l'identité d'un utilisateur qui tente d'accéder à un système.

Autorisation : L'autorisation détermine les ressources auxquelles un utilisateur authentifié peut accéder, en fonction de ses rôles et permissions

4. Mise en Œuvre de l'Authentification et de l'Autorisation pour Sécuriser l'API

Authentification : Pour sécuriser l'API avec l'authentification, suivez ces étapes :

- Créer un contrôleur avec un endpoint `/api/auth/login` qui accepte les identifiants de l'utilisateur.
- Dans ce contrôleur, vérifier les identifiants, et si corrects, générer un JWT et le retourner.

Autorisation : Pour mettre en œuvre l'autorisation, suivez ces étapes :

- Définir des rôles utilisateur et administrateur dans le modèle utilisateur.
- Assigner les rôles appropriés aux utilisateurs lors de leur création ou mise à jour.

Compréhension des Business Requirements (fonctionnels et non-fonctionnels)

Exigences Fonctionnelles :

Gestion des Utilisateurs :

- Création, mise à jour, suppression et consultation des utilisateurs.
- Authentification des utilisateurs via un end point de connexion.
- Gestion des rôles et permissions des utilisateurs.

Gestion des Véhicules :

- Ajout, mise à jour, suppression et consultation des véhicules.
- Association des véhicules avec les utilisateurs.

Exigences Non-Fonctionnelles :

Performance :

- L'API doit répondre aux requêtes en moins de 200ms en moyenne.
- Supporter jusqu'à 1000 requêtes simultanées.

Sécurité :

- Protection des end points sensibles via JWT.
- Cryptage des mots de passe des utilisateurs.

Scalabilité :

- Supporter l'ajout de nouveaux micro services sans affecter les performances.
- Déploiement facile avec Docker et orchestration via Docker Compose.

Maintenance :

- Code bien documenté et structuré pour faciliter les mises à jour et les corrections de bugs.
- Tests unitaires et d'intégration automatisés.

Identification de la cible de l'application

Public Cible :

➤ Développeurs de Propelize:

- Utiliseront l'API pour intégrer des fonctionnalités de gestion des utilisateurs et des véhicules dans diverses applications front-end.

➤ Administrateurs :

- Géreront les utilisateurs et les rôles via des outils administratifs utilisant l'API.

➤ **Utilisateurs Finaux :**

- Accéderont aux services de gestion des véhicules via des applications front-end sécurisées par l'API.

Etablissement des objectifs de cette application

- Fournir une API sécurisée pour la gestion des utilisateurs et des véhicules.
- Permettre une authentification et une autorisation robustes basées sur JWT.
- Faciliter le déploiement et l'intégration des services via Docker Compose.
- Garantir la performance et la scalabilité des micro services.

Exploration du Processus de Conception

a. **Analyse des Exigences :**

- Identification des besoins des utilisateurs et des administrateurs.
- Définition des fonctionnalités essentielles et des priorités.

b. **Conception de l'Architecture :**

- Définition des micro services (API Gateway, Service Utilisateur, Service Véhicule).
- Définition des communications entre les services (REST).

c. **Sélection des Technologies :**

- Java Springs Boot pour le développement des micro services.
- JWT pour l'authentification.
- MySQL pour la base de données.
- Docker et Docker Compose pour le déploiement.

Identification des Ressources et des Actions

Actions :

➤ Utilisateurs :

- Créer, lire, mettre à jour, supprimer.

➤ Véhicules :

- Ajouter, lire, mettre à jour, supprimer.
- Associer un utilisateur à un véhicule.
- Authentification (login).
- Rafraîchir le token d'accès.

Le Choix des Design Principles (REST, GraphQL, SOAP)

REST :

➤ Avantages :

- Simplicité et flexibilité.
- Utilisation des méthodes HTTP standard (GET, POST, PUT, DELETE).
- Facilité d'intégration avec les clients existants.

Choix Justification :

- **Simplicité** : REST est bien adapté pour les opérations CRUD (Create, Read, Update, Delete) nécessaires pour la gestion des utilisateurs et des véhicules.
- **Compatibilité** : REST est largement supporté et facilite l'intégration avec divers clients front-end et autres services.

La Définition des End Points

Utilisateurs :

- **POST /api/users:** Créer un utilisateur.
- **GET /api/users/{id}:** Récupérer un utilisateur par ID.
- **PUT /api/users/{id}:** Mettre à jour un utilisateur.
- **DELETE /api/users/{id}:** Supprimer un utilisateur.

Véhicules :

- **POST /api/vehicles:** Ajouter un véhicule.
- **GET /api/vehicles/{id}:** Récupérer un véhicule par ID.
- **PUT /api/vehicles/{id}:** Mettre à jour un véhicule.
- **DELETE /api/vehicles/{id}:** Supprimer un véhicule.
- **POST /api/auth/login:** Authentifier un utilisateur.
- **POST /api/auth/refresh:** Rafraîchir le token JWT.
-

Considérations sur la Sécurité

Authentification :

- Utilisation de JWT pour les requêtes sécurisées.
- Stockage sécurisé des mots de passe avec bcrypt.

Autorisation :

- Contrôle d'accès basé sur les rôles (RBAC) avec Spring Security.
- Utilisation d'annotations pour restreindre l'accès aux endpoints sensibles.